LeetCode Pattern Recognition Guide

Tailored for MLOps / ML Engineer — Big Tech (FAANG) Entry Level

This guide covers the 12 core problem-solving patterns that appear in 95%+ of ML Engineer coding interviews at Big Tech companies. For each pattern you get: when to use it, how to recognize it from the problem statement, the template code, and the exact problems to practice.

Your Interview Scope (What to Focus On)

Based on your target role (MLOps/ML Engineer, fresher, Big Tech), here is exactly what the coding round looks like:

Category	Expectation	Priority
Difficulty	Easy + Medium only. No Hard DP.	✅ Required
Problem count	1 problem per round (45 min)	✅ Required
Patterns needed	12 core patterns (this guide)	✅ Required
Advanced DP	Bitmask, complex interval DP	❌ Skip for ML Engineer
Graph algorithms	BFS, DFS only. Not Dijkstra.	⚠️ Basic only
Segment trees, Tries	Rarely tested for ML roles	❌ Low priority

Pattern Recognition Master Table

When you see these keywords in a problem, map them to a pattern immediately:

If you see...	Use this pattern	Difficulty
sorted array + pair/triplet	**Two Pointers**	Easy
longest/shortest subarray/substring	**Sliding Window**	Medium
linked list cycle / middle	**Fast & Slow Pointers**	Easy
find pair / count frequency / duplicates	**HashMap / HashSet**	Easy
sorted array + search / find boundary	**Binary Search**	Easy–Medium
shortest path / minimum steps / level order	**BFS**	Medium

all paths / connected components / tree traversal	**DFS**	**Medium**
K largest / K smallest / streaming min-max	**Heap**	**Medium**
max/min value, number of ways, overlapping subproblems	**Dynamic Programming**	**Medium**
all combinations / permutations / subsets	**Backtracking**	**Medium**
overlapping intervals / meeting rooms	**Merge Intervals**	**Medium**
next greater element / valid brackets / histogram	**Monotonic Stack**	**Medium**

The 12 Patterns — Deep Dive

Pattern 01: Two Pointers

When to use	Keyword signals	Complexity
Sorted array/string. Finding pairs, triplets, or subarrays that meet a condition. Removing duplicates in-place.	*"Find pair that sums to X"* | *"Remove duplicates"* | *"Is palindrome"* | *"Container with most water"*	**Time: O(n) | Space: O(1)**

Approach:

Left pointer at start, right at end (or both at start for fast/slow). Move based on condition.

Code Template (Python):

```
left, right = 0, len(arr) - 1
while left < right:
    if arr[left] + arr[right] == target:
        return [left, right]
    elif arr[left] + arr[right] < target:
        left += 1
    else:
        right -= 1
```

Practice Problems (NeetCode 75 mapped):

- Two Sum II (sorted)
- Valid Palindrome
- 3Sum
- Container With Most Water
- Remove Duplicates

Pattern 02: Sliding Window

When to use	Keyword signals	Complexity
Contiguous subarray/substring. Finding max/min length window satisfying a condition. Substring problems.	*"Longest substring with..."* | *"Maximum sum subarray of size k"* | *"Minimum window substring"*	**Time: O(n) | Space: O(k) where k = unique elements**

Approach:

Expand right pointer. Shrink left when window condition violated. Track best result.

Code Template (Python):

```
left = 0
window = {}  # or Counter()
result = 0
for right in range(len(s)):
    window[s[right]] = window.get(s[right], 0) + 1
    while <window_invalid_condition>:
        window[s[left]] -= 1
        left += 1
    result = max(result, right - left + 1)
```

Practice Problems (NeetCode 75 mapped):

- Longest Substring Without Repeating
- Maximum Sum Subarray of Size K
- Fruit Into Baskets
- Minimum Window Substring

Pattern 03: Fast & Slow Pointers

When to use	Keyword signals	Complexity
Linked list cycle detection. Finding middle of linked list. Detecting loops.	*"Detect cycle"* | *"Find middle"* | *"Happy number"* | *"Find duplicate"*	**Time: O(n) | Space: O(1)**

Approach:

slow moves 1 step, fast moves 2 steps. If they meet → cycle. When fast hits end → slow is at middle.

Code Template (Python):

```
slow, fast = head, head
```

```
while fast and fast.next:
    slow = slow.next
    fast = fast.next.next
    if slow == fast:
        return True  # cycle detected
return False
```

Practice Problems (NeetCode 75 mapped):

- Linked List Cycle
- Find Middle of Linked List
- Happy Number
- Find the Duplicate Number

Pattern 04: HashMap / HashSet

When to use	Keyword signals	Complexity
Counting frequencies. Looking up existence in O(1). Grouping elements. Two-pass problems.	*"Find duplicate" | "Group anagrams" | "Subarray sum equals k" | "Two sum"*	**Time: O(n) | Space: O(n)**

Approach:

Store seen values or counts in dict. For complement problems: check if target - current exists in map.

Code Template (Python):

```
seen = {}
for i, num in enumerate(nums):
    complement = target - num
    if complement in seen:
        return [seen[complement], i]
    seen[num] = i
```

Practice Problems (NeetCode 75 mapped):

- Two Sum
- Group Anagrams
- Top K Frequent Elements
- Subarray Sum Equals K
- Valid Anagram

Pattern 05: Binary Search

When to use	Keyword signals	Complexity

Sorted array. Finding a value or boundary. "Find minimum in rotated array". Answer-space search.	*"Search in sorted" | "Find minimum/maximum" | "Kth smallest" | array is sorted or rotated*	**Time: O(log n) | Space: O(1)**

Approach:

mid = left + (right - left) // 2. Eliminate half based on condition. Track boundary carefully.

Code Template (Python):

```
left, right = 0, len(nums) - 1
while left <= right:
    mid = left + (right - left) // 2
    if nums[mid] == target:
        return mid
    elif nums[mid] < target:
        left = mid + 1
    else:
        right = mid - 1
return -1
```

Practice Problems (NeetCode 75 mapped):

- Binary Search
- Search in Rotated Sorted Array
- Find Minimum in Rotated Array
- Koko Eating Bananas

Pattern 06: BFS (Breadth-First Search)

When to use	Keyword signals	Complexity
Shortest path in unweighted graph/grid. Level-order traversal. Minimum steps problems.	*"Shortest path" | "Minimum steps" | "Level order" | "Rotten oranges" | "Word ladder"*	**Time: O(V + E) | Space: O(V)**

Approach:

Use deque. Process level by level. Track visited to avoid cycles. Count levels for distance.

Code Template (Python):

```
from collections import deque
queue = deque([start])
visited = {start}
steps = 0
while queue:
    for _ in range(len(queue)):  # process level
```

```
        node = queue.popleft()
        for neighbor in get_neighbors(node):
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
    steps += 1
```

Practice Problems (NeetCode 75 mapped):

- Binary Tree Level Order Traversal
- Rotting Oranges
- Number of Islands
- Word Ladder
- Shortest Path in Grid

Pattern 07: DFS (Depth-First Search)

When to use	Keyword signals	Complexity
Exploring all paths. Tree/graph traversal. Connected components. Backtracking problems.	*"All paths"* | *"Number of islands"* | *"Path sum"* | *"Clone graph"* | *"Flood fill"*	**Time: O(V + E) | Space: O(V) recursion stack**

Approach:

Recursive or stack. Mark visited before recursing. Unmark if backtracking (permutations).

Code Template (Python):

```
def dfs(node, visited):
    if node in visited:
        return
    visited.add(node)
    # process node
    for neighbor in graph[node]:
        dfs(neighbor, visited)

visited = set()
dfs(start, visited)
```

Practice Problems (NeetCode 75 mapped):

- Number of Islands
- Max Area of Island
- Path Sum
- Clone Graph
- Course Schedule

Pattern 08: Heap / Priority Queue

When to use	Keyword signals	Complexity
Finding K largest/smallest. Streaming data min/max. Merge K sorted lists.	*"K largest"* | *"K most frequent"* | *"Median of stream"* | *"Merge K sorted"*	**Time: O(n log k) | Space: O(k)**

Approach:

Python heapq is a min-heap. Negate values for max-heap. Push/pop is O(log n).

Code Template (Python):

```
import heapq

# Min-heap (default)
heap = []
heapq.heappush(heap, val)
min_val = heapq.heappop(heap)

# Max-heap (negate values)
heapq.heappush(heap, -val)
max_val = -heapq.heappop(heap)

# K largest elements
return heapq.nlargest(k, nums)
```

Practice Problems (NeetCode 75 mapped):

- Kth Largest Element
- Top K Frequent Elements
- Find Median From Data Stream
- Merge K Sorted Lists

Pattern 09: Dynamic Programming (1D)

When to use	Keyword signals	Complexity
Optimization with overlapping subproblems. "Number of ways". Max/min value problems.	*"Number of ways to..."* | *"Maximum profit"* | *"Climb stairs"* | *"Coin change"* | *"House robber"*	**Time: O(n) | Space: O(n) or O(1) with optimization**

Approach:

Define dp[i] = answer for subproblem i. Find recurrence relation. Fill bottom-up.

Code Template (Python):

```
# House Robber pattern
dp = [0] * len(nums)
dp[0] = nums[0]
dp[1] = max(nums[0], nums[1])
for i in range(2, len(nums)):
    dp[i] = max(dp[i-1],        # skip current
                dp[i-2] + nums[i])  # rob current
return dp[-1]

# Space optimized:
prev2, prev1 = 0, 0
for num in nums:
    prev2, prev1 = prev1, max(prev1, prev2 + num)
```

Practice Problems (NeetCode 75 mapped):

- Climbing Stairs
- House Robber
- Coin Change
- Longest Increasing Subsequence
- Jump Game

Pattern 10: Backtracking

When to use	Keyword signals	Complexity
Generate all combinations/permutations/subsets. Constraint satisfaction. N-Queens type problems.	*"All combinations" | "All permutations" | "All subsets" | "Generate valid parentheses"*	**Time: O(2^n) or O(n!) | Space: O(n) depth**

Approach:

Choose → Explore → Unchoose. Prune branches early. Use index to avoid duplicate combos.

Code Template (Python):

```
def backtrack(start, current):
    result.append(list(current))  # record state
    for i in range(start, len(nums)):
        current.append(nums[i])   # choose
        backtrack(i + 1, current) # explore
        current.pop()             # unchoose

result = []
backtrack(0, [])
return result
```

Practice Problems (NeetCode 75 mapped):

- Subsets
- Permutations
- Combination Sum

- Generate Parentheses
- Letter Combinations

Pattern 11: Merge Intervals

When to use	Keyword signals	Complexity
Overlapping intervals. Scheduling problems. Meeting rooms.	*"Merge overlapping" | "Insert interval" | "Meeting rooms" | "Employee free time"*	**Time: O(n log n) | Space: O(n)**

Approach:

Sort by start time. Compare current start with previous end. Merge if overlapping.

Code Template (Python):

```
intervals.sort(key=lambda x: x[0])
merged = [intervals[0]]
for start, end in intervals[1:]:
    if start <= merged[-1][1]:  # overlap
        merged[-1][1] = max(merged[-1][1], end)
    else:
        merged.append([start, end])
return merged
```

Practice Problems (NeetCode 75 mapped):

- Merge Intervals
- Insert Interval
- Meeting Rooms II
- Non-overlapping Intervals

Pattern 12: Stack (Monotonic)

When to use	Keyword signals	Complexity
Next greater/smaller element. Temperature problems. Histogram problems.	*"Next greater element" | "Daily temperatures" | "Largest rectangle" | "Valid parentheses"*	**Time: O(n) | Space: O(n)**

Approach:

Maintain stack in increasing/decreasing order. Pop when current element breaks the order.

Code Template (Python):

```
# Next Greater Element (monotonic decreasing stack)
```

```
stack = []  # stores indices
result = [-1] * len(nums)
for i, num in enumerate(nums):
    while stack and nums[stack[-1]] < num:
        idx = stack.pop()
        result[idx] = num  # found next greater
    stack.append(i)
return result
```

Practice Problems (NeetCode 75 mapped):

- Daily Temperatures
- Next Greater Element
- Largest Rectangle in Histogram
- Valid Parentheses

4-Week Study Plan (2–3 problems/day)

This plan covers NeetCode 75 in 4 weeks at a relaxed pace — appropriate for ML Engineer prep alongside your distributed systems syllabus.

Week	Patterns	Daily Load	Goal
Week 1	HashMap, Two Pointers, Sliding Window	2 problems/day	Build core intuition on easiest patterns
Week 2	Binary Search, Fast & Slow, Merge Intervals	2 problems/day	Sorted array + linked list confidence
Week 3	BFS, DFS, Stack	2–3 problems/day	Graph traversal + tree fluency
Week 4	Heap, DP, Backtracking	3 problems/day	Optimization + enumeration problems

Interview Tips for ML Engineer Coding Round

The coding round for ML Engineer is not just about getting the right answer. Communication and pattern recognition speed matter as much.

Tip	Why it matters
Think out loud immediately	Say "This looks like a sliding window problem because we need a contiguous subarray." Interviewers reward pattern recognition.
State the pattern before coding	Spend 2–3 minutes identifying the pattern and confirming your approach before writing a single line.

Write brute force first if stuck	A working O(n²) solution is better than an unfinished O(n) one. Then optimize.
Always mention complexity	After coding, always state time and space complexity unprompted. This is expected.
Use Python for speed	Python is faster to write and interviewers accept it. Use built-ins (Counter, deque, heapq) without hesitation.
Edge cases to always check	Empty input, single element, all same elements, negative numbers, very large inputs.

Remember: System Design > LeetCode for ML Engineer roles at Big Tech.

Master these 12 patterns, then invest the majority of your remaining prep time in ML system design and distributed systems.